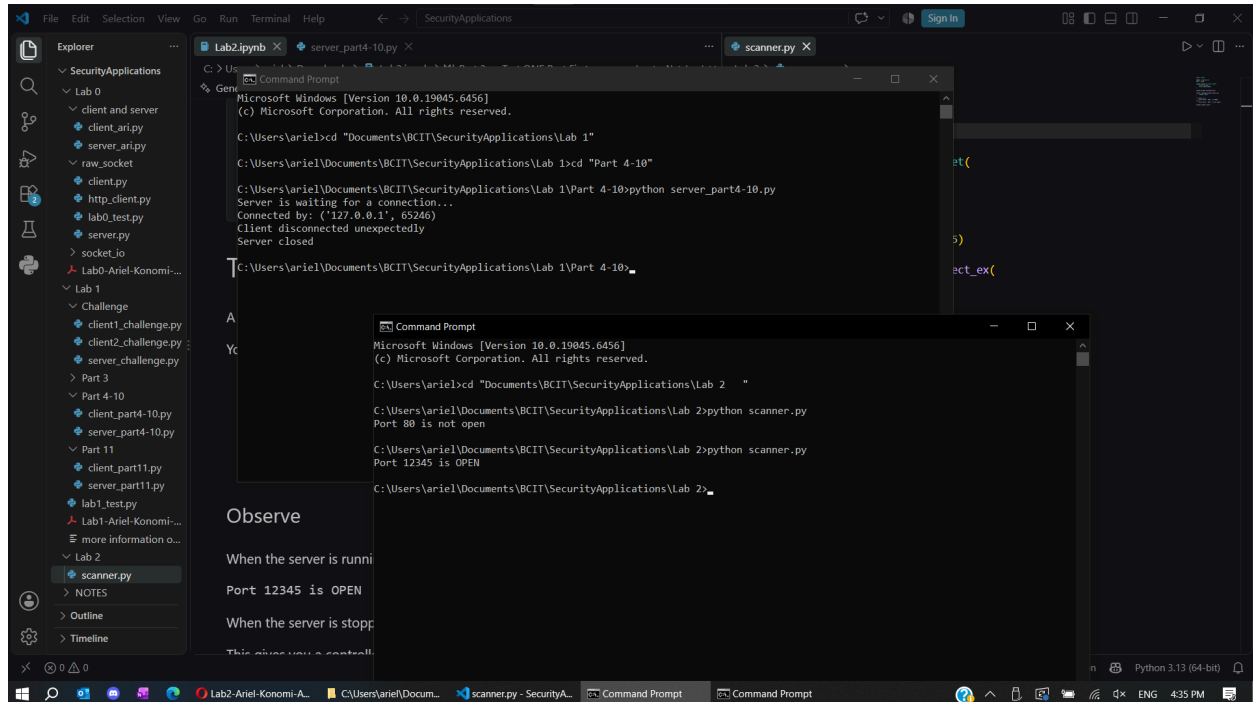


Lab 2: Developing Python Nmap Port Scanner

Part 3



Part 4

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ariel>cd "Documents\BCIT\SecurityApplications\Lab 1"

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1>cd "Part 4-10"

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 65246)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
```

```
scanner.py
1 import socket
2
3 def scan_port(target, port):
4
5     sock = socket.socket(
6         socket.AF_INET,
7         socket.SOCK_STREAM
8     )
9
10    sock.settimeout(0.5)
11
12    result = sock.connect_ex(
13        (target, port)
14    )
15
16    sock.close()
17
18    if result == 0:
19        return True
20    else:
21        return False
22
23 print(scan_port("127.0.0.1", 80))
```

```
Microsoft Windows [Version 10.0.19045.6456]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ariel>cd "Documents\BCIT\SecurityApplications\Lab 1"

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1>cd "Part 4-10"

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 65246)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 65229)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>
```

```
scanner.py
1 import socket
2
3 def scan_port(target, port):
4
5     sock = socket.socket(
6         socket.AF_INET,
7         socket.SOCK_STREAM
8     )
9
10    sock.settimeout(0.5)
11
12    result = sock.connect_ex(
13        (target, port)
14    )
15
16    sock.close()
17
18    if result == 0:
19        return True
20    else:
21        return False
22
23 print(scan_port("127.0.0.1", 12345))
```

Part 5

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 49327)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 49812)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Port 12345 is OPEN

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Port 12345 is OPEN
Open ports: [12345]

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

```
scanner.py
1 import socket
2
3 def scan_port(target, port):
4     sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5     sock.settimeout(0.5)
6
7     result = sock.connect_ex((target, port))
8     sock.close()
9
10    if result == 0:
11        return True
12    else:
13        return False
14
15 target = "127.0.0.1"
16 start_port = 12340
17 end_port = 12350
18
19 open_ports = []
20
21 for port in range(start_port, end_port + 1):
22     if scan_port(target, port):
23         print("Port", port, "is OPEN")
24         open_ports.append(port)
25
26 if len(open_ports) > 0:
27     print("Open ports:", open_ports)
28 else:
29     print("No open ports found in the selected range.")
```

Part 6

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 49327)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 49812)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Port 12345 is OPEN

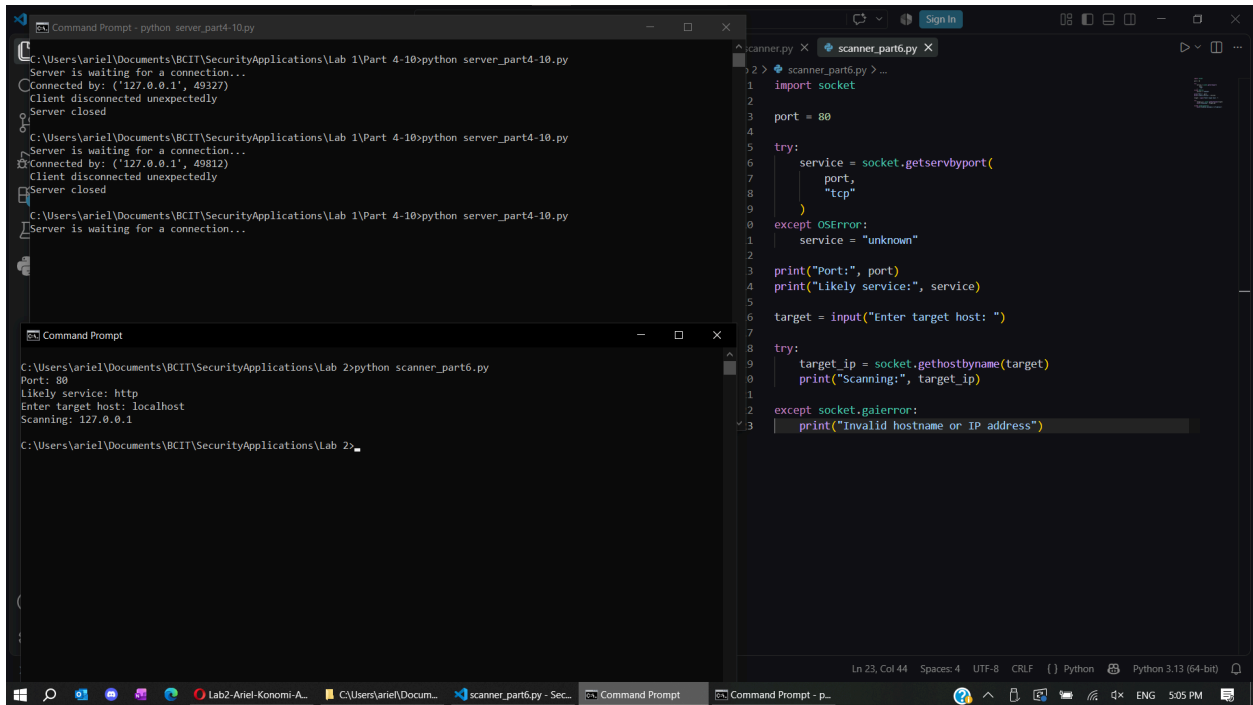
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Port 12345 is OPEN
Open ports: [12345]

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner_part6.py
Port: 80
Likely service: http

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

```
scanner_part6.py
1 import socket
2
3 port = 80
4
5 try:
6     service = socket.getservbyport(
7         port,
8         "tcp"
9     )
10 except OSError:
11     service = "unknown"
12
13 print("Port:", port)
14 print("Likely service:", service)
```

Part 7



The screenshot shows a Windows desktop with two windows. The top window is a Command Prompt titled 'python_server_part4-10.py' showing the execution of a server script. The bottom window is a Command Prompt titled 'Command Prompt' showing the execution of a scanner script. The scanner script is also visible in a separate window titled 'scanner_part6.py'.

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 49327)
Client disconnected unexpectedly
Server closed

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
Connected by: ('127.0.0.1', 49812)
Client disconnected unexpectedly
Server closed

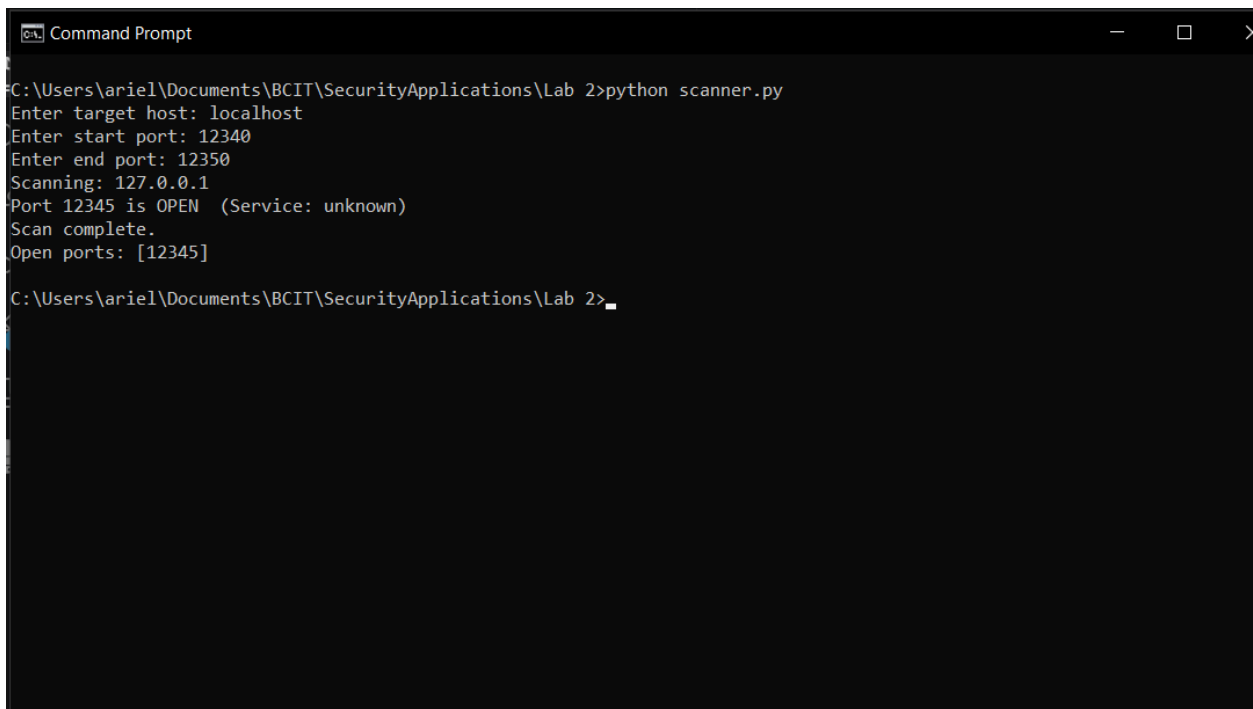
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 1\Part 4-10>python server_part4-10.py
Server is waiting for a connection...
```

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner_part6.py
Port: 80
Likely service: http
Enter target host: localhost
Scanning: 127.0.0.1

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

```
scanner.py scanner_part6.py
1 import socket
2
3 port = 80
4
5 try:
6     service = socket.getservbyport(
7         port,
8         "tcp"
9     )
10 except OSError:
11     service = "unknown"
12
13 print("Port:", port)
14 print("Likely service:", service)
15
16 target = input("Enter target host: ")
17
18 try:
19     target_ip = socket.gethostbyname(target)
20     print("Scanning:", target_ip)
21
22 except socket.gaierror:
23     print("Invalid hostname or IP address")
```

Part 9



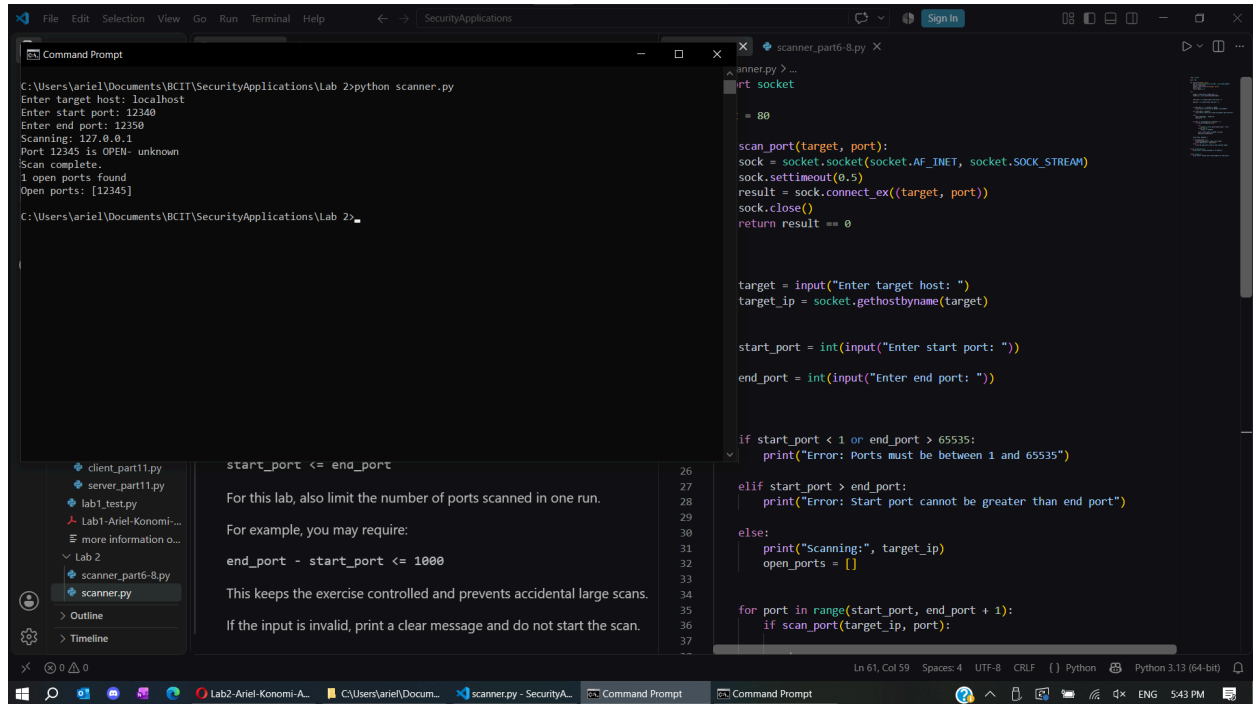
The screenshot shows a Windows Command Prompt window titled 'Command Prompt' with the following output:

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 12340
Enter end port: 12350
Scanning: 127.0.0.1
Port 12345 is OPEN (Service: unknown)
Scan complete.
Open ports: [12345]

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

Part 10

Known open port



The screenshot shows a Visual Studio Code editor with a Python script named `scanner.py` and a Command Prompt window running the script.

Command Prompt Output:

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 12340
Enter end port: 12350
Scanning: 127.0.0.1
Port 12345 is OPEN- unknown
Scan complete.
1 open ports found
Open ports: [12345]

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

Python Script (scanner.py):

```
import socket

def scan_port(target, port):
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(0.5)
    result = sock.connect_ex((target, port))
    sock.close()
    return result == 0

target = input("Enter target host: ")
target_ip = socket.gethostbyname(target)

start_port = int(input("Enter start port: "))
end_port = int(input("Enter end port: "))

if start_port < 1 or end_port > 65535:
    print("Error: Ports must be between 1 and 65535")
elif start_port > end_port:
    print("Error: Start port cannot be greater than end port")
else:
    print("Scanning:", target_ip)
    open_ports = []

    for port in range(start_port, end_port + 1):
        if scan_port(target_ip, port):
```

Known closed port

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 12340
Enter end port: 12350
Scanning: 127.0.0.1
Port 12345 is OPEN- unknown
Scan complete.
1 open ports found
Open ports: [12345]

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 150
Enter end port: 165
Scanning: 127.0.0.1
Scan complete.
No open ports found in the selected range

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

Record the results.

Compare

Answer:

1. Did Nmap identify the same open ports as your Python scanner?
2. What additional information did Nmap provide?
3. Did Nmap distinguish states that your scanner could not?
4. Why is your Python scanner useful for learning even though

```
scanner_part6-8.py
...
return result == 0

target = input("Enter target host: ")
target_ip = socket.gethostbyname(target)

start_port = int(input("Enter start port: "))
end_port = int(input("Enter end port: "))

if start_port < 1 or end_port > 65535:
    print("Error: Ports must be between 1 and 65535")

if start_port > end_port:
    print("Error: Start port cannot be greater than end port")

else:
    print("Scanning:", target_ip)
    open_ports = []

    for port in range(start_port, end_port + 1):
        if scan_port(target_ip, port):
            try:
                service = socket.getservbyport(port, "tcp")
            except OSError:
                service = "unknown"

            print ("Port",port,"is OPEN-",service)
            open_ports.append(port)

    print("Scan complete.")
```

Invalid host name

```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 12340
Enter end port: 12350
Scanning: 127.0.0.1
Port 12345 is OPEN- unknown
Scan complete.
1 open ports found
Open ports: [12345]

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 150
Enter end port: 165
Scanning: 127.0.0.1
Scan complete.
No open ports found in the selected range

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: invalid-test
Error: Invalid hostname or IP address

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

Record the results.

Compare

Answer:

1. Did Nmap identify the same open ports as your Python scanner?
2. What additional information did Nmap provide?
3. Did Nmap distinguish states that your scanner could not?
4. Why is your Python scanner useful for learning even though

```
scanner_part6-8.py
...
return result == 0

target = input("Enter target host: ")
target_ip = socket.gethostbyname(target)

start_port = int(input("Enter start port: "))
end_port = int(input("Enter end port: "))

if start_port < 1 or end_port > 65535:
    print("Error: Ports must be between 1 and 65535")

if start_port > end_port:
    print("Error: Start port cannot be greater than end port")

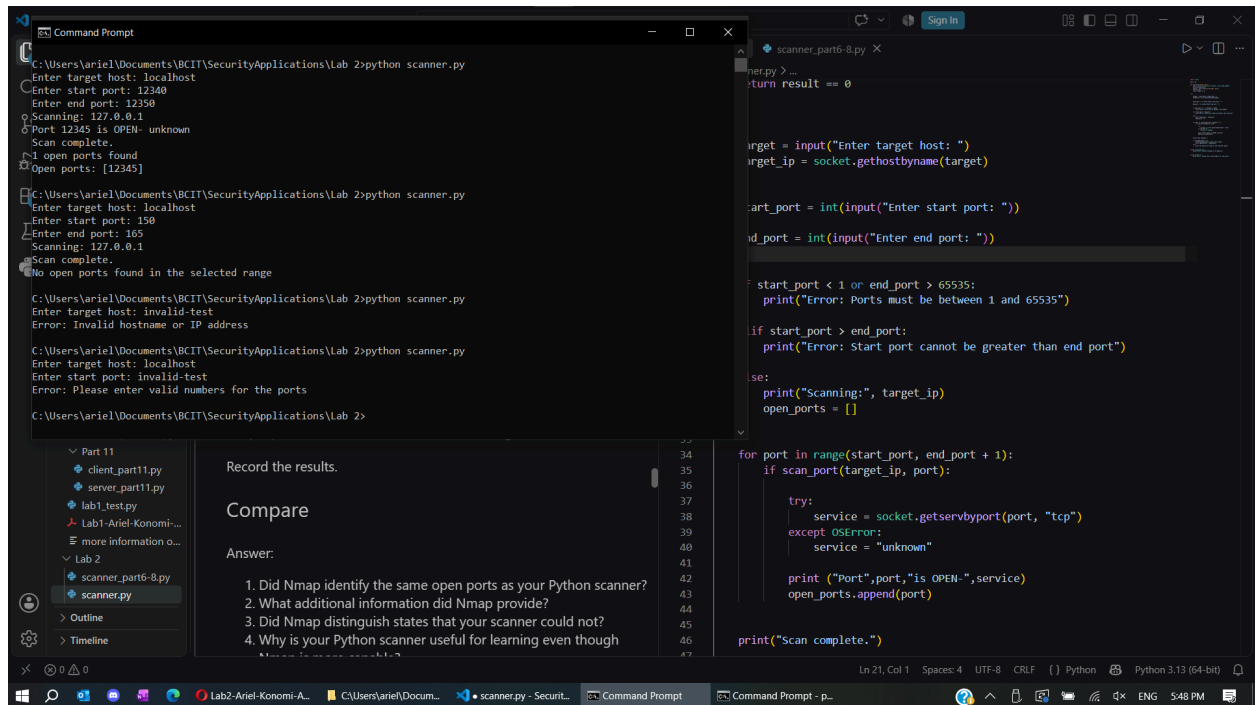
else:
    print("Scanning:", target_ip)
    open_ports = []

    for port in range(start_port, end_port + 1):
        if scan_port(target_ip, port):
            try:
                service = socket.getservbyport(port, "tcp")
            except OSError:
                service = "unknown"

            print ("Port",port,"is OPEN-",service)
            open_ports.append(port)

    print("Scan complete.")
```

Invalid port input (non numeric)



```
C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 12340
Enter end port: 12350
Scanning: 127.0.0.1
Port 12345 is OPEN- unknown
Scan complete.
1 open ports found
Open ports: [12345]

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: 150
Enter end port: 165
Scanning: 127.0.0.1
Scan complete.
No open ports found in the selected range

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: invalid-test
Error: Invalid hostname or IP address

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>python scanner.py
Enter target host: localhost
Enter start port: invalid-test
Error: Please enter valid numbers for the ports

C:\Users\ariel\Documents\BCIT\SecurityApplications\Lab 2>
```

```
scanner_part6_8.py
...
return result == 0

...

target = input("Enter target host: ")
target_ip = socket.gethostbyname(target)

start_port = int(input("Enter start port: "))
end_port = int(input("Enter end port: "))

if start_port < 1 or end_port > 65535:
    print("Error: Ports must be between 1 and 65535")

if start_port > end_port:
    print("Error: Start port cannot be greater than end port")

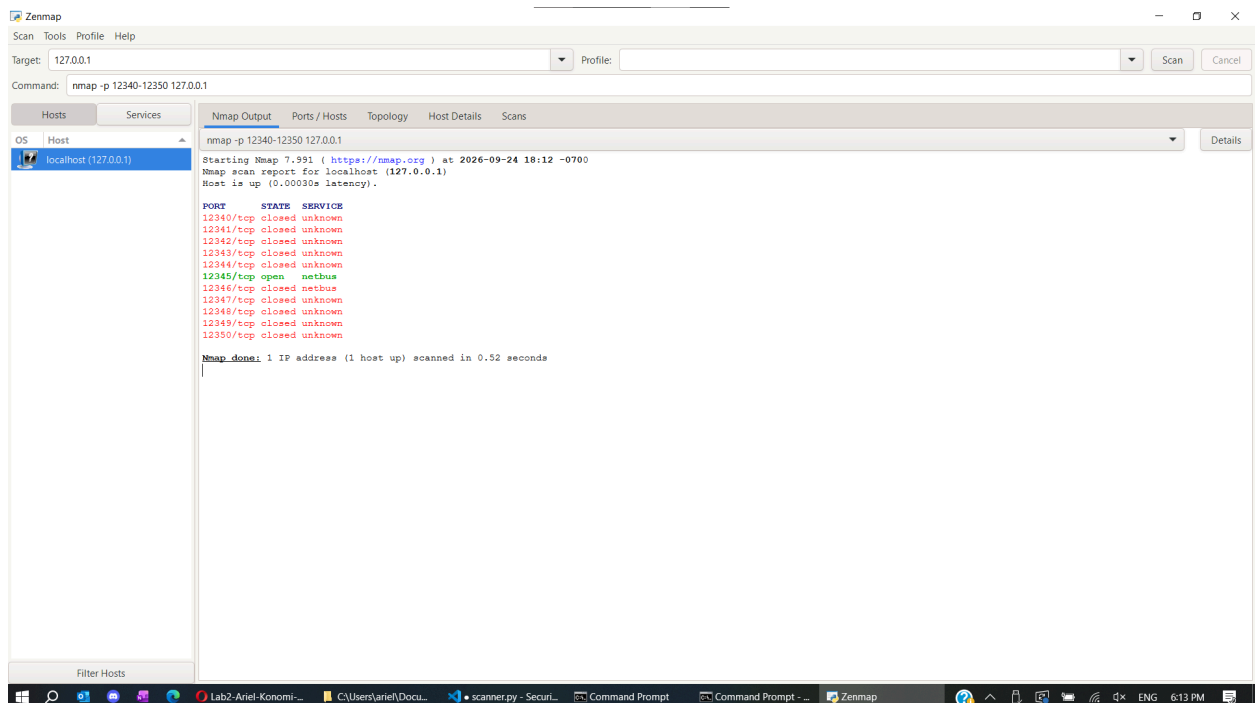
else:
    print("Scanning:", target_ip)
    open_ports = []

    for port in range(start_port, end_port + 1):
        if scan_port(target_ip, port):
            try:
                service = socket.getservbyport(port, "tcp")
            except OSError:
                service = "unknown"

            print ("Port",port,"is OPEN-",service)
            open_ports.append(port)

    print("Scan complete.")
```

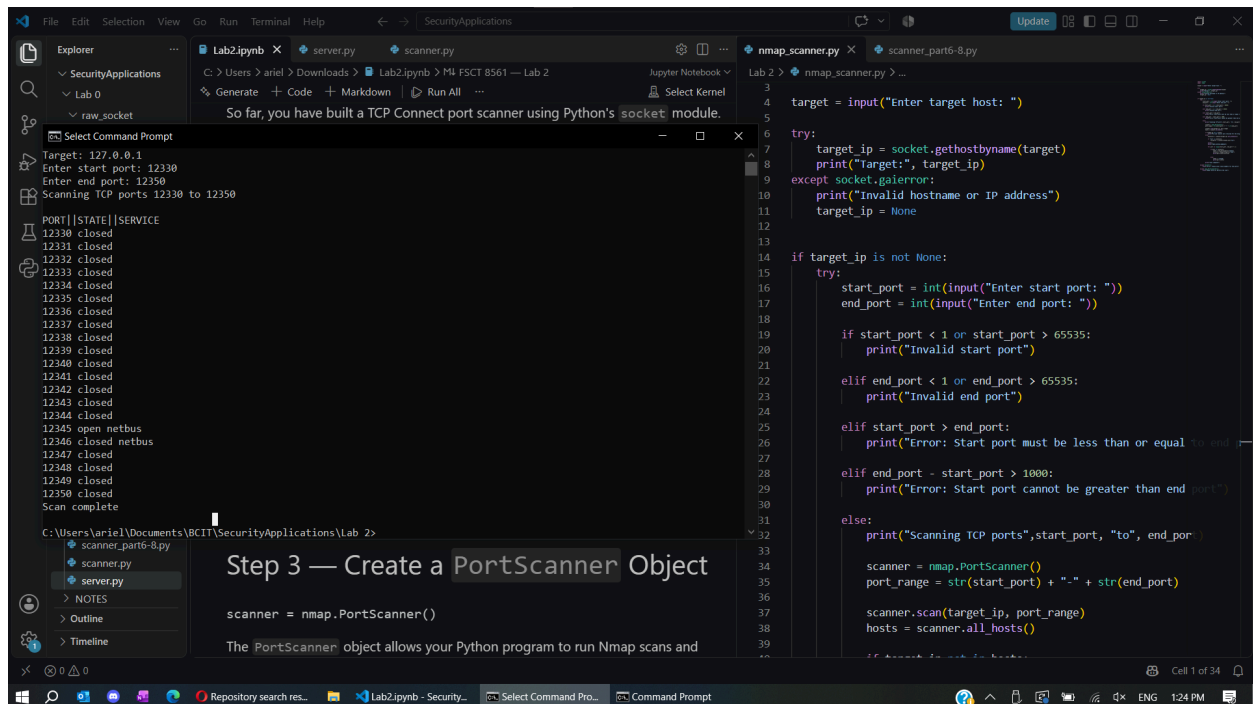
Part 11



Questions

1. Did Nmap identify the same open ports as your Python scanner?
 - a. Yes, nmap did identify the same open ports as my python scanner
2. What additional information did Nmap provide?
 - a. nmap shows all of the ports within the range given, and shows whether they are tcp or udp. It also showed the service used by the open port
3. Did Nmap distinguish states that your scanner could not?
 - a. nmap was able to distinguish whether the ports were open or not, basically giving a reason as to why it could not connect to that port
4. Why is your Python scanner useful for learning even though Nmap is more capable?
 - a. By creating the python scanner, you can learn how the process works/ is created in the background. You can connect sockets, ports, loops etc. to build a basic scanner

Part 12



The screenshot shows a Jupyter Notebook interface with the following components:

- File Explorer (Left):** Shows the project structure with 'SecurityApplications' containing 'Lab2.ipynb' and 'raw_socket'.
- Terminal Output (Bottom Left):** Displays the results of a scan on target 127.0.0.1, showing ports 12330 through 12350. Most ports are closed, while ports 12345 and 12346 are open and identified as 'netbus'.
- Code Cell (Right):** Contains Python code for creating a PortScanner object. The code includes input prompts for target host, start port, and end port, and logic to validate these inputs before scanning.
- Step 3 — Create a PortScanner Object:** A section header with a code snippet: `scanner = nmap.PortScanner()`. Below it, a note states: 'The PortScanner object allows your Python program to run Nmap scans and'.

Questions

1. Which version required more code?
 - a. The raw socket scanner requires more code as you are not relying on importing nmap for the scans
2. Which version provided more information?
 - a. The nmap version provided more information as it shows the scan results from nmap itself
3. What information did Nmap provide that your raw socket scanner did not?
 - a. Nmap provides all ports within the range, shows which are opened and closed, and also shows the service of each port
4. What work is `python-nmap` hiding from the programmer?
 - a. Python-nmap hides the background process of nmap and only shows the results
5. Why was it useful to build the raw socket version **before** using `python-nmap`?
 - a. It was useful to build the raw socket version before as you can learn how the scanning works. After that, a base has basically been created/established for python-nmap
6. If you were developing a real security application, which approach would you choose and why?
 - a. I would choose the python-nmap approach simply because writing and creating a manual script has a higher chance of having mistakes. Using nmap would also ensure that a known scanner that works would be implemented

Part 13

1. What information can an attacker learn from port scanning?
 - a. Attackers can learn which ports are open and if they accept connections. They can also learn what services are being used per port.
2. Why can an open port increase a system's attack surface?
 - a. An open port can increase a system's attack surface as it means that an application or network service is listening for connections. If the port is not secured, then it would be a direct path for attackers to use maliciously
3. Does an open port automatically mean that a vulnerability exists? Explain.
 - a. An open port does not automatically mean that a vulnerability exists. If the port has been secured from known vulnerabilities, it should be safe from attackers
4. Why should unnecessary network services be disabled or restricted?
 - a. Unnecessary network services should be disabled or restricted to minimize the exposure to malicious attacks.
5. What is the difference between an open port and a closed port?

- a. An open port is a port that is listening and accepting connections. A closed port is a port that is not accepting any connections to it.
- 6. Why might a firewall make scan results difficult to interpret?
 - a. A firewall might make scan results difficult as it uses filters. It would make the results difficult to interpret as it would not specifically mean that a port is closed, only that a connection has been blocked on said port
- 7. Why can our simple TCP Connect scanner not reliably identify every filtered port?
 - a. The scanner that we created is a simple TCP scanner. It only verifies/checks whether a connection is successful, not whether a port is closed or filtered
- 8. How could defenders detect or limit scanning activity?
 - a. Defenders detect or limit scanning activity by actively monitoring network traffic and using firewalls to restrict access
- 9. Why should defenders scan their own systems?
 - a. Defenders should scan their own systems to verify exposed services and identify exposures
- 10. Why is authorization important before performing network scanning?
 - a. Authorization is important before performing network scanning to ensure that only authorized users can access this important information

Part 14

I built and executed a scanner for TCP ports, to see which ports within a certain range were open. The target that I was authorized to use was localhost (127.0.0.1). The port range tested was 12330-12350 during the different parts of the lab. Both of the scanners used during this lab (self created scanner and python-nmap) showed a single port open at TCP 12345. The rest of the ports within the range refused connections. For this specific port, the service associated with it was netbus.

Normally any port that is open means that they are actively listening for a connection. This does not mean that the port is automatically vulnerable, but generally, open ports can create unmonitored entry points. Attackers can leverage these entry points to map the network, identify software versions as well as launch exploits against misconfigurations or unpatched vulnerabilities

I built the raw socket TCP scanner before I implemented nmap. This was important in learning how a scanner can be programmed and how it works in the background. I learned how a basic TCP scanner utilizes `socket.connect_ex()`, and used timeouts to ensure that the program would

not listen infinitely. When using nmap, however, it showed how simple and limited the raw socket TCP scanner was at the beginning. Nmap showed the whole range of ports, and displayed whether they were open/closed as well as the service these ports used.

To reduce the unnecessary attack surface, defensive measures must be applied throughout the implemented system. Ports that are not used or not authorized need to be closed to limit the attack surface and ensure that ports are not actively listening for connections. Firewall rules should be implemented as well, to drop any unwanted inbound traffic. This would shield the system from external probing. This lab demonstrated how port scanning provides useful information regarding the security of a system's network and how it can be used to lower the attack surface of it.